

P0440r0 : Floating Point Atomic View

Project: ISO JTC1/SC22/WG21: Programming Language C++
Number: P0440r0
Date: 2016-10-14
Reply-to: hcedwar@sandia.gov
Author: H. Carter Edwards
Contact: hcedwar@sandia.gov
Author: Hans Boehm
Contact: hboehm@google.com
Author: Olivier Giroux
Contact: ogiroux@nvidia.com
Author: JF Bastien
Contact: jfbastien@apple.com
Author: James Reus
Contact: reus1@llnl.gov
Audience: SG1 Concurrency, Library Evolution
URL: <https://github.com/kokkos/ISO-CPP-Papers/blob/master/P0440.rst>

Revision History

P0440r0

- Apply Floating Point Atomic (P0020) to Atomic View (P0019).

Overview / Motivation

This paper proposes an extension to the atomic operations library [atomics] for atomic addition on an object conforming to **atomic_view<T>** (P0019) where T is a *floating-point* type (N5131 3.9.1p8). This capability is critical for high performance computing (HPC) applications.

Naming

Naming will conform to the to-be-determined naming convention selected for atomic view (P0019). The two currently proposed atomic view naming conventions are as follows.

```
namespace std {  
namespace experimental {  
    template<> struct atomic_view< floating-point >;  
}}
```

OR

```
namespace std {  
namespace experimental {  
    template<> struct atomic< floating-point & >;  
}}
```

Proposal

add to 29.2 Header <atomic> synopsis

```
namespace std {  
    template<> struct atomic_view< floating-point >;  
}
```

add to 29.5 Atomic Types

```
template<> struct atomic_view<floating-point> {
    static constexpr size_t required_alignment = implementation-defined ;
    static constexpr bool is_always_lock_free = implementation-defined ;
    bool is_lock_free() const noexcept;
    void store( floating-point , memory_order = memory_order_seq_cst ) const noexcept;
    floating-point load( memory_order = memory_order_seq_cst ) const noexcept;
    operator floating-point () const noexcept ;
    floating-point exchange( floating-point , memory_order = memory_order_seq_cst ) const noexcept;
    bool compare_exchange_weak( floating-point & , floating-point , memory_order , memory_order ) const noexcept;
    bool compare_exchange_strong( floating-point & , floating-point , memory_order , memory_order ) const noexcept;
    bool compare_exchange_weak( floating-point & , floating-point , memory_order = memory_order_seq_cst ) const
    noexcept;
    bool compare_exchange_strong( floating-point & , floating-point , memory_order = memory_order_seq_cst ) const
    noexcept;

    floating-point fetch_add( floating-point , memory_order = memory_order_seq_cst ) const noexcept;
    floating-point fetch_sub( floating-point , memory_order = memory_order_seq_cst ) const noexcept;

    constexpr atomic_view() noexcept ;
    atomic_view( atomic_view && ) noexcept ;
    atomic_view( const atomic_view & ) noexcept ;
    atomic & operator = ( atomic_view && ) noexcept ;
    atomic & operator = ( const atomic_view & ) noexcept ;
    floating-point operator=( floating-point ) noexcept ;

    explicit atomic_view( floating-point & obj ) noexcept ;
    explicit constexpr operator bool () const noexcept;

    floating-point operator+=( floating-point ) const ;
    floating-point operator-=( floating-point ) const ;
};
```

in 29.6.6 Requirements for operations on atomic view types

regarding arithmetic operations

C A::fetch_key(M operand, memory_order order = memory_order_seq_cst) const noexcept;

update remark as follows

Remark: For signed integer types, arithmetic is defined to use two's complement representation. ~~There and there~~ are no undefined results. For floating point types, if the result is not mathematically defined or not in the range of representable values for its type (5p4) the result is unspecified. [Note: Atomic arithmetic operations on *floating-point* should conform to **std::numeric_limits<floating-point>** traits associated with the floating point type (18.3.2). The floating point environment (26.4) for atomic arithmetic operations on *floating-point* may be different than the calling thread's floating point environment. - end note] For address types, the result may be an undefined address, but the operations otherwise have no undefined behavior.